

24.两两交换链表中的节点

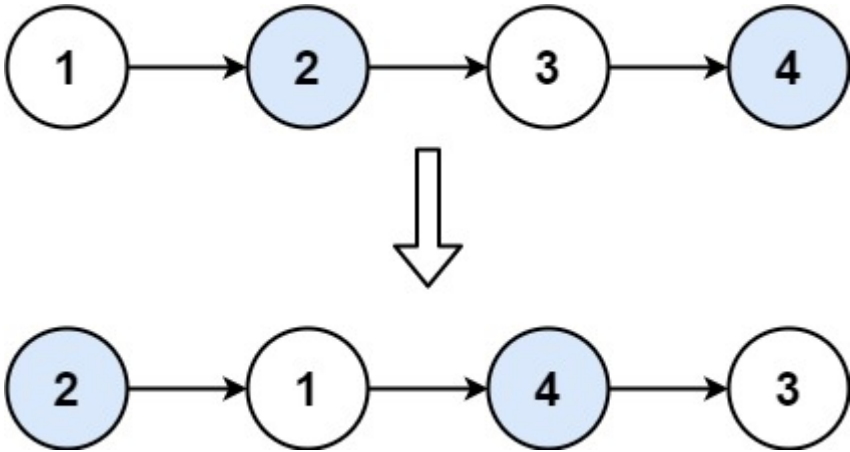
两两交换链表中的节点

Category	Difficulty	Likes	Dislikes	ContestSlug	ProblemIndex	Score
algorithms	Medium (74.08%)	2403	0	-	-	0

- Tags
- Companies

给你一个链表，两两交换其中相邻的节点，并返回交换后链表的头节点。你必须在不修改节点内部的值的情况下完成本题（即，只能进行节点交换）。

示例 1:



- ```
1 输入: head = [1,2,3,4]
2 输出: [2,1,4,3]
```

示例 2:

- ```
1 输入: head = []
2 输出: []
```

示例 3:

- ```
1 输入: head = [1]
2 输出: [1]
```

提示:

- 链表中节点的数目在范围 `[0, 100]` 内
- `0 <= Node.val <= 100`

## 2.方法

# LeetCode 24: 两两交换链表中的节点（详细解析）

## 1. 问题理解

**目标：** 给定一个单链表，将其中的节点**两两交换**。

**输入：** 单链表的头节点 `head`

**输出：** 两两交换节点后的链表的头节点

**示例：**

- 输入： `head = [1,2,3,4]`
- 输出： `[2,1,4,3]`
- 输入： `head = [1,2,3]`
- 输出： `[2,1,3]` (最后一个节点保持不变)

**约束条件：**

- 不能只是单纯地改变节点内部的值，而是需要实际进行节点交换（即修改节点的 `next` 指针）。
- 空间复杂度应为  $O(1)$ （如果使用迭代法）。

## 2. 基本思路

我们需要遍历链表，每次处理一对节点，交换它们的位置。

主要方法：

1. **迭代法** - 使用循环，每次处理一对节点，需要仔细管理指针。
2. **递归法** - 定义一个函数，该函数负责交换一对节点，并递归调用自身处理剩余部分。

## 3. 解法一：迭代法（推荐）

### 思路

**核心思想：** 为了方便地交换节点并连接交换后的部分，我们需要一个指针指向**待交换的两个节点的前一个节点**。同时，使用哑节点可以简化对头两个节点的处理。

**指针管理：**

- `dummy`: 哑节点, `dummy.next = head`。
- `prev`: 指向**待交换的两个节点的前一个节点**。初始时 `prev = dummy`。
- `node1`: 指向待交换的第一个节点 (`prev.next`)。
- `node2`: 指向待交换的第二个节点 (`prev.next.next`)。

**交换步骤（对于每一对 `node1`, `node2`）：**

1. 将 `prev` 的 `next` 指向 `node2`。

2. 将 `node1` 的 `next` 指向 `node2` 的 `next` (连接到下一对或链表末尾)。
3. 将 `node2` 的 `next` 指向 `node1`。
4. 更新 `prev` 指针, 为下一轮交换做准备。交换后, `node1` 变成了这对节点的第二个节点, 所以 `prev` 应该移动到 `node1` 的位置。

## 图解过程

假设链表为 `dummy -> 1 -> 2 -> 3 -> 4`, 当前 `prev = dummy`:

### 1. 识别节点:

- `prev = dummy`
- `node1 = 1`
- `node2 = 2`

### 2. 执行交换:

- `prev.next = node2` ( `dummy.next = 2` )

```

1 | dummy -> 2
2 | ^
3 | 1 -> 2 -> 3 -> 4
4 |

```

- `node1.next = node2.next` ( `1.next = 3` )

```

1 | dummy -> 2
2 | ^ \
3 | 1 -----> 3 -> 4
4 |

```

- `node2.next = node1` ( `2.next = 1` )

```

1 | dummy -> 2 -> 1 -> 3 -> 4
2 |

```

- 链表变为: `dummy -> 2 -> 1 -> 3 -> 4`

### 3. 更新 `prev`:

- `prev = node1` ( `prev` 移动到节点 1 的位置)

### 4. 下一轮循环:

- `prev = 1`
- `node1 = 3`
- `node2 = 4`
- ...重复交换步骤...

## 代码实现

```
1 4.swap-nodes-in-pairs.java
2 /**
3 * Definition for singly-linked list.
4 * public class ListNode {
5 * int val;
6 * ListNode next;
7 * ListNode() {}
8 * ListNode(int val) { this.val = val; }
9 * ListNode(int val, ListNode next) { this.val = val; this.next = next;
10 }
11 * }
12 */
13 class Solution {
14 /**
15 * 两两交换链表中的节点 - 迭代法
16 * @param head 链表头节点
17 * @return 交换后的链表头节点
18 */
19 public ListNode swapPairs(ListNode head) {
20 // 创建哑节点，简化头两个节点的交换处理
21 ListNode dummy = new ListNode(0);
22 dummy.next = head;
23
24 // prev 指向待交换两个节点的前一个节点
25 ListNode prev = dummy;
26
27 // 循环条件：确保 prev 后面至少还有两个节点可以交换
28 while (prev.next != null && prev.next.next != null) {
29 // 获取待交换的两个节点
30 ListNode node1 = prev.next;
31 ListNode node2 = prev.next.next;
32
33 // 执行交换
34 // 1. prev 指向 node2
35 prev.next = node2;
36 // 2. node1 指向 node2 的下一个节点
37 node1.next = node2.next;
38 // 3. node2 指向 node1
39 node2.next = node1;
40
41 // 更新 prev 指针，移动到下一对要交换的节点之前
42 // 交换后，node1 变成了这对节点的第二个节点
43 prev = node1;
44 }
45
46 // 返回哑节点的下一个节点，即交换后的新头节点
47 return dummy.next;
48 }
49 }
```

## 复杂度分析

- **时间复杂度：**  $O(N)$ ，其中  $N$  是链表的长度。需要遍历链表一次。
- **空间复杂度：**  $O(1)$ ，只使用了常数个额外指针变量。

## 4. 解法二：递归法

### 思路

**核心思想：** 定义一个递归函数，该函数负责交换当前头两个节点，并将其余部分交给递归调用处理。

**步骤：**

1. **基本情况：** 如果链表为空或只有一个节点，无需交换，直接返回 `head`。
2. **递归步骤：**
  - a. 获取前两个节点： `node1 = head`, `node2 = head.next`。
  - b. 递归调用 `swapPairs` 处理从 `node2.next` 开始的剩余链表，得到交换后的剩余部分的头节点 `swappedRest`。
  - c. 执行当前这对节点的交换：
    - i. `node2.next = node1` (第二个节点指向第一个节点)
    - ii. `node1.next = swappedRest` (第一个节点指向后面已交换好的部分)
  - d. 返回 `node2` 作为当前已交换部分的新头节点。

### 代码实现

```
1 4.swap-nodes-in-pairs.java
2 /**
3 * Definition for singly-linked list.
4 * public class ListNode {
5 * int val;
6 * ListNode next;
7 * ListNode() {}
8 * ListNode(int val) { this.val = val; }
9 * ListNode(int val, ListNode next) { this.val = val; this.next = next;
10 }
11 * }
12 */
13 class Solution {
14 /**
15 * 两两交换链表中的节点 - 递归法
16 * @param head 链表头节点
17 * @return 交换后的链表头节点
18 */
19 public ListNode swapPairs(ListNode head) {
20 // 基本情况：链表为空或只有一个节点，无需交换
21 if (head == null || head.next == null) {
22 return head;
23 }
24
25 // 获取前两个节点
26 ListNode node1 = head;
27 ListNode node2 = head.next;
```

```

27
28 // 递归交换 node2 后面的节点
29 ListNode swappedRest = swapPairs(node2.next);
30
31 // 执行当前这对节点的交换
32 node2.next = node1; // 第二个节点指向第一个节点
33 node1.next = swappedRest; // 第一个节点指向后面已交换好的部分
34
35 // node2 现在是交换后这对节点的新头节点
36 return node2;
37 }
38 }
39

```

## 复杂度分析

- **时间复杂度：**  $O(N)$ ，每个节点被访问和处理一次。
- **空间复杂度：**  $O(N)$ ，递归调用会产生调用栈，最坏情况下栈的深度为  $N$ 。

## 5. 如何在面试中讲解

### 1. 理解问题和约束：

- "目标是两两交换链表中的节点，需要实际修改节点的 `next` 指针，而不是只改值。"
- "需要处理空链表、单节点链表以及节点数为奇数的情况。"

### 2. 提出主要方法：

- "这个问题可以用迭代或递归来解决。迭代法通常空间复杂度更优 ( $O(1)$ )，而递归法代码可能更简洁但空间复杂度为  $O(N)$ 。我先讲解迭代法。"

### 3. 讲解迭代法：

- "为了方便处理头节点的交换，我们可以引入一个哑节点 `dummy`，让 `dummy.next` 指向 `head`。"
- "我们需要一个 `prev` 指针，它始终指向待交换的两个节点的前一个节点。初始时 `prev` 指向 `dummy`。"
- "我们循环遍历链表，条件是 `prev` 后面至少还有两个节点 (`prev.next != null && prev.next.next != null`)。"
- "在循环内部，我们定义 `node1 = prev.next` 和 `node2 = prev.next.next`。"
- "然后执行交换的三个关键步骤："
  - "`prev.next = node2` (将前驱节点连接到第二个节点)"
  - "`node1.next = node2.next` (将第一个节点连接到后续链表)"
  - "`node2.next = node1` (将第二个节点连接到第一个节点)"
- "完成交换后，需要更新 `prev` 指针，让它移动到下一对要交换的节点之前。因为 `node1` 现在是这对节点的第二个，所以 `prev = node1`。"
- "循环结束后，返回 `dummy.next`。"

### 4. 分析迭代法复杂度：

- "时间复杂度是  $O(N)$ ，因为我们遍历链表一次。"
- "空间复杂度是  $O(1)$ ，只用了常数个额外指针。"

#### 5. (可选) 简述递归法：

- "递归的思路是：定义一个函数，如果链表少于两个节点，直接返回。否则，先递归交换除了头两个节点之外的剩余部分，然后交换头两个节点，并将第一个节点连接到递归返回的结果上，最后返回第二个节点作为新的头。"
- "递归法代码简洁，但空间复杂度是  $O(N)$  因为递归栈。"

#### 6. 处理边界情况：

- "迭代法中的哑节点很好地处理了头节点交换的情况。"
- "循环条件 `prev.next != null && prev.next.next != null` 自然地处理了空链表、单节点链表以及节点数为奇数的情况（最后一轮循环不会执行）。"

选择迭代法进行详细讲解通常更受面试官青睐，因为它展示了对指针操作的熟练掌握和对空间复杂度的关注。

找到具有 1 个许可证类型的类似代码